



Home Work

DevOps

HW_L5_01_Kubernetes

۱. فایل تمرین را در پنل خود آپلود کنید.

۲. title فایل تمرین به صورت (نام تمرین+نام و نام خانوادگی) به انگلیسی باشد.

نمونه: AmirMohammadMohammadi_kubernetes_01

۳. در صورتی که سوال و یا ابهامی دارید در گروه چت تلگرامی بپرسید.

تمرین 1 Kubernetes

مجموع امتیاز: چهل نمره

این تمرین دو محور دارد:

۱. مقدمه — Kubernetes درک «چرا» orchestration و «معماری» cluster

۲. مرور — Docker تثبیت مفاهیم و دستورات پایه قبل از کار عمیق تر با Kubernetes

نمره	بخش‌ها	محور
۱۰	۰۲،۰۱	الف — مقدمه Kubernetes
۲۰	۰۶،۰۵،۰۴،۰۳	ب — مبانی Docker
۱۰	۰۸،۰۷	ج Docker — چندسرویی

شرایط اجرا و تحویل

- محیط Ubuntu 22.04، Debian، یا WSL/VM با دسترسی `sudo`
- برای هر بخش، نام پوشه و فایل را دقیقاً مطابق سوال بسازید
- تحویل: یک پوشه فشرده شامل تمام زیرپوشه‌ها، اسکریپت‌ها، کانفیگ‌ها و خروجی دستورها
- اسکریپت‌ها را قابل اجرا کنید `chmod +x *.sh` :
- پیش‌نیاز: آشنایی با خط فرمان Linux؛ Docker در بخش ۰۳ نصب یا بررسی می‌شود

ساختار پوشه کار:

kubernetes_homework/

└── 01_kubernetes_why/

└── 02_kubernetes_architecture/

- |— 03_docker_concepts/
- |— 04_docker_images/
- |— 05_docker_containers/
- |— 06_docker_dockerfile/
- |— 07_docker_volumes_networks/
- |— 08_docker_compose/

بخش الف — مقدمه Kubernetes

۰۱ — چرا Container Orchestration؟ — پنج نمره

هدف: درک مشکلات مدیریت دستی container و نقش orchestrator

۱.۱ مشکلات قبل از — Orchestration ۲.۵ نمره

فایل 01_kubernetes_why/problems_before_orchestration.md بسازید و پاسخ دهید:

۱. حداقل ۴ مشکل اجرای اپلیکیشن فقط با docker run (چند ماشین) بدون orchestrator

۲. Self-healing در Kubernetes یعنی چه؟ — یک مثال عملی

۳. تفاوت Rolling Update با deploy دستی (توقف همه → اجرای همه)

۴. چرا Docker به تنهایی برای production در مقیاس بزرگ کافی نیست؟

Hints:

- به crash ، scale ، load balancing و downtime فکر کنید

۱.۲ مقایسه Orchestrator ها — ۲.۵ نمره

فایل 01_kubernetes_why/orchestrators_comparison.md بسازید:

۱. جدول مقایسه **Kubernetes**، **Docker Swarm**، **Nomad** (حداقل ۳ معیار)

۲. دو دلیل محبوبیت Kubernetes در صنعت

۳. یک سناریو واقعی (مثلاً افزایش ناگهانی ترافیک) — بدون orchestrator چه می شود و با Kubernetes چه کمکی می شود؟

۰۲ — معماری — Kubernetes پنج نمره

هدف: شناخت Control Plane ، Node و جریان اجرای یک Pod

۲.۱ اجزای Cluster — ۳ نمره

فایل — 02_kubernetes_architecture/components.md برای هر مورد نقش + یک جمله مثال:

کامپوننت	محل
API Server	Control Plane
etcd	Control Plane
Scheduler	Control Plane
Controller Manager	Control Plane
kubelet	Node
kube-proxy	Node
Container Runtime	Node

فایل — 02_kubernetes_architecture/request_flow.txt جریان زیر را در ۵ تا ۷ مرحله شماره گذاری کنید:

Pod → `kubectl apply -f deployment.yaml` روی Node در حال اجرا

Hints:

- API Server نقطه ورود همه درخواست‌هاست
- Scheduler فقط Node مناسب را انتخاب می‌کند؛ اجرا با kubelet است

۲.۲ بررسی محیط — `kubectl` ۲ نمره

```
#!/bin/bash
```

```
set -e
```

```
kubectl version --client > kubectl_version.txt 2>&1 || echo "kubectl not installed" > kubectl_version.txt
```

```
kubectl cluster-info >> kubectl_version.txt 2>&1 || true
```

```
kubectl get nodes -o wide > nodes.txt 2>&1 || echo "no cluster available" > nodes.txt
```

```
kubectl get --raw='/readyz' > cluster_health.txt 2>&1 || true
```

اگر **cluster** ندارید: فایل 02_kubernetes_architecture/setup_notes.txt بنویسید — چرا
cluster در دسترس نیست و چه مراحل برای راه اندازی لوکال (minikube) یا (kind) لازم است.
حداقل kubectl version --client در kubectl_version.txt الزامی است.

Hints:

- minikube start یا kind create cluster برای cluster لوکال
- بدون cluster ، بخش تئوری همچنان قابل ارزیابی است

بخش ب — مبانی Docker

۰۳ — مفاهیم و نصب — Docker پنج نمره

هدف: تفاوت VM و Container؛ اطمینان از آماده بودن Docker

۳.۱ مفاهیم پایه — ۲ نمره

فایل: 03_docker_concepts/concepts.md

۱. تفاوت VM و Container (حداقل ۴ معیار)

۲. تفاوت Image و Container

۳. اجزای اصلی (Docker Engine خلاصه، daemon، CLI، runtime)

۳.۲ بررسی نصب — ۳ نمره

اسکریپت: 03_docker_concepts/docker_check.sh

```
#!/bin/bash
```

```
set -e
```

```
docker --version > docker_version.txt
```

```
docker info 2>/dev/null | head -n 25 >> docker_version.txt || echo "docker  
daemon not running" >> docker_version.txt
```

```
docker ps -a > docker_ps.txt
```

```
systemctl is-active docker 2>/dev/null > docker_service.txt || echo "n/a" >  
docker_service.txt
```

فایل — 03_docker_concepts/status_summary.txt در حداکثر ۵ خط: وضعیت نصب، وضعیت سرویس، تعداد containers

Hints:

- نصب: `sudo apt update && sudo apt install -y docker.io`
- اجرا بدون sudo: `sudo usermod -aG docker $USER` نیاز به (logout/login)

۰۴ — Image — پنج نمره

هدف: pull، tag، inspect و درک لایه‌ها

۴.۱ عملیات — Image ۳ نمره

اسکرپت: 04_docker_images/image_ops.sh

```
#!/bin/bash
```

```
set -e
```

```
docker pull nginx:alpine
```

```
docker images | grep nginx > image_list.txt
```

```
docker tag nginx:alpine my-nginx:v1
```

```
docker images | grep -E 'nginx|my-nginx' >> image_list.txt
```



```
docker inspect nginx:alpine --format '{{.Architecture}} {{.Os}}' >  
image_inspect.txt
```

```
docker history nginx:alpine --no-trunc | head -n 15 > image_history.txt
```

```
docker rmi my-nginx:v1 2>/dev/null || true
```

خروجی تمام فایل‌های تولیدشده را در پوشه تحویل دهید.

۴.۲ خلاصه لایه‌ها — ۲ نمره

فایل 04_docker_images/layers_summary.txt:

۱. تقریباً چند layer در history دیده می‌شود؟

۲. چرا image های alpine کوچک‌ترند؟

۳. تفاوت docker pull و docker build

۵ — Container — پنج نمره

هدف: اجرا، متغیر محیطی، port mapping، exec

۵.۱ اجرا و Environment — ۲.۵ نمره

اسکرپت: 05_docker_containers/container_basics.sh

```
#!/bin/bash
```

```
set -e
```

```
docker rm -f lab-nginx lab-env 2>/dev/null || true
```

```
docker run -d --name lab-nginx nginx:alpine
```

```
docker ps --filter name=lab-nginx --format "table  
{{.Names}}\t{{.Status}}\t{{.Ports}}" > container_ps.txt
```

```
docker logs lab-nginx 2>&1 | head -n 8 > container_logs.txt
```

```
docker run -d --name lab-env -e APP_NAME=lab -e PORT=8080 alpine sleep  
3600
```

```
docker exec lab-env env | grep -E 'APP_NAME|PORT' > env_vars.txt
```

```
docker stop lab-nginx lab-env && docker rm lab-nginx lab-env
```

۵.۲ Port Mapping و exec — ۲.۵ نمره

اسکرپت: 05_docker_containers/port_and_exec.sh

```
#!/bin/bash
```

```
set -e
```

```
docker rm -f lab-web 2>/dev/null || true
```

```
docker run -d --name lab-web -p 9080:80 nginx:alpine
```

```
sleep 2
```

```
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:9080 > curl_status.txt
```

```
curl -s http://localhost:9080 | head -n 5 > curl_body.txt
```

```
docker port lab-web >> curl_status.txt
```

```
docker exec lab-web sh -c 'echo lab-test > /tmp/test.txt'
```

```
docker exec lab-web cat /tmp/test.txt > exec_result.txt
```

```
docker stop lab-web && docker rm lab-web
```

فایل — 05_docker_containers/exec_vs_attach.txt
docker exec تفاوت و docker
(attach ۳-۵ خط)

Hints:

- اگر پورت ۹۰۸۰ اشغال است، پورت دیگر انتخاب کنید و در curl_status.txt یادداشت کنید

۰۶ — Dockerfile — پنج نمره

هدف: ساخت image سفارشی و درک دستورات Dockerfile

۶.۱ پروژه — build ۵ نمره

ساختار:

06_docker_dockerfile/

└─ app/

└─ server.py

|— Dockerfile

|— .dockerignore

|— build_and_run.sh

|— dockerfile_notes.txt

app/server.py:

```
from http.server import BaseHTTPRequestHandler, HTTPServer
```

```
class Handler(BaseHTTPRequestHandler):
```

```
    def do_GET(self):
```

```
        self.send_response(200)
```

```
        self.send_header("Content-type", "text/plain")
```

```
        self.end_headers()
```

```
        self.wfile.write(b"Hello from custom image\n")
```

```
if __name__ == "__main__":
```

```
    HTTPServer(("0.0.0.0", 8000), Handler).serve_forever()
```

Dockerfile:

```
FROM python:3.11-alpine
```

WORKDIR /app

COPY app/server.py .

EXPOSE 8000

CMD ["python", "server.py"]

.dockerignore:

*.md

notes.txt

build_and_run.sh:

#!/bin/bash

set -e

cd "\$(dirname "\$0")"

docker build -t lab-python-server:1.0 . 2>&1 | tee build_log.txt

docker images | grep lab-python-server > built_images.txt

docker run -d --name lab-py -p 8000:8000 lab-python-server:1.0

sleep 2

curl -s http://localhost:8000 > curl_app.txt

docker history lab-python-server:1.0 --no-trunc | head -n 12 >
layers_after_build.txt

```
docker stop lab-py && docker rm lab-py
```

— dockerfile_notes.txt نقش هر

دستور FROM ، COPY ، EXPOSE ، CMD یک خط برای هر کدام)

Hints:

- هر خط Dockerfile یک layer جدید می‌سازد

بخش ج Docker — چند سرویسی

۷ Volume — و Network پنج نمره

هدف: ذخیره‌سازی پایدار و ارتباط بین container ها

۷.۱ Volume — ۲.۵ نمره

اسکرپت: 07_docker_volumes_networks/volume_demo.sh

```
#!/bin/bash
```

```
set -e
```

```
cd "$(dirname "$0")"
```

```
docker volume create lab-data
```

```
docker run --rm -v lab-data:/data alpine sh -c 'echo persistent > /data/msg.txt'
```

```
docker run --rm -v lab-data:/data alpine cat /data/msg.txt > volume_persist.txt
```

```
docker volume inspect lab-data > volume_inspect.txt
```

```
mkdir -p web
```

```
echo '<h1>Bind mount test</h1>' > web/index.html
```

```
docker run -d --name lab-bind -p 9090:80 \
```

```
-v "$(pwd)/web:/usr/share/nginx/html:ro" nginx:alpine
```

```
sleep 2
```

```
curl -s http://localhost:9090 | head -n 5 > bind_mount_curl.txt
```

```
docker stop lab-bind && docker rm lab-bind
```

۷.۲ — Network ۲.۵ نمره

اسکرپت: 07_docker_volumes_networks/network_demo.sh

```
#!/bin/bash
```

```
set -e
```

```
docker network create lab-net 2>/dev/null || true
```

```
docker rm -f lab-a lab-b 2>/dev/null || true
```

```
docker run -d --name lab-a --network lab-net alpine sleep 3600
```

```
docker run -d --name lab-b --network lab-net alpine sleep 3600
```

```
docker exec lab-a ping -c 2 lab-b > ping_by_name.txt
```

```
docker network inspect lab-net > network_inspect.txt
```

```
docker rm -f lab-a lab-b
```

فایل 07_docker_volumes_networks/summary.txt:

۱. تفاوت bind mount و named volume

۲. چرا در شبکه سفارشی با نام container می‌توان ارتباط برقرار کرد؟

۸ — Docker Compose — پنج نمره

هدف: تعریف چند سرویس در یک فایل و اجرای همزمان

۸.۱ پروژه — Compose ۵ نمره

ساختار:

```
08_docker_compose/
```

```
├── docker-compose.yml
```

```
├── web/
```

```
│   └── index.html
```

```
├── compose_demo.sh
```

```
└── compose_summary.txt
```

web/index.html:


```
<!DOCTYPE html>
```

```
<html><body><h1>Compose Lab</h1></body></html>
```

```
docker-compose.yml:
```

```
services:
```

```
  web:
```

```
    image: nginx:alpine
```

```
    ports:
```

```
      - "8088:80"
```

```
    volumes:
```

```
      - ./web:/usr/share/nginx/html:ro
```

```
    depends_on:
```

```
      - api
```

```
  api:
```

```
    image: alpine
```

```
    command: sh -c "while true; do sleep 30; done"
```

```
compose_demo.sh:
```

```
#!/bin/bash
```

```
set -e
```

```
cd "$(dirname "$0")"
```

```
docker compose down -v 2>/dev/null || docker-compose down -v 2>/dev/null  
|| true
```

```
docker compose up -d 2>&1 | tee compose_up.txt || docker-compose up -d  
2>&1 | tee compose_up.txt
```

```
sleep 4
```

```
docker compose ps > compose_ps.txt 2>&1 || docker-compose ps >  
compose_ps.txt
```

```
curl -s http://localhost:8088 | head -n 5 > compose_web.txt
```

```
docker compose exec web ping -c 2 api > compose_ping.txt 2>&1 \
```

```
|| docker-compose exec web ping -c 2 api > compose_ping.txt
```

```
docker network ls > compose_networks.txt
```

```
docker compose down 2>&1 | tee compose_down.txt || docker-compose down  
2>&1 | tee compose_down.txt
```

```
compose_summary.txt:
```

۱. `depends_on` چه کاری می‌کند؟

۲. سرویس `web` چگونه به `api` در شبکه داخلی دسترسی دارد؟

۳. چه زمانی `docker compose` به جای `docker run` مناسب‌تر است؟

تحويل نهایی

نام گذاری

[نام_خانوادگی]_kubernetes_01/

└─ kubernetes_homework/

└─ 01_kubernetes_why/

└─ ...

└─ 08_docker_compose/

فایل های ریشه پوشه تحويل

فایل

محتوا

README.md

ساختار پوشه، پیش نیازها، نحوه اجرای اسکریپت ها

final_structure.txt

خروجی find tree kubernetes_homework
kubernetes_homework

commands_history.txt

دستورات مهم docker و kubectl

فشرده سازی

zip -r [نام_خانوادگی]_kubernetes_01.zip [نام_خانوادگی]_kubernetes_01/

معیارهای امتیازدهی

توزیع نمره (۴۰)

نمره	عنوان	بخش
۵	چراOrchestration	۰۱
۵	معماریKubernetes	۰۲
۵	مفاهیم و نصب Docker	۰۳
۵	Image	۰۴
۵	Container	۰۵
۵	Dockerfile	۰۶
۵	Network و Volume	۰۷
۵	Docker Compose	۰۸

معیار هر بخش

سهم تقریبی

معیار

۲۰٪	رعایت نام پوشه و فایل
۵۰٪ (بخش‌های عملی)	اجرای صحیح و خروجی قابل بررسی
۳۰٪	کیفیت پاسخ‌های مفهومی

معیار کل تمرین

سهم	معیار
۴۰٪	صحت فنی و خروجی‌ها
۳۵٪	درک مفاهیم
۱۵٪	رعایت ساختار و نام فایل‌ها
۱۰٪	README و نظم تحویل

نکات

۱. پیشوند -lab در نام container ها در اسکریپت‌ها استفاده شده تا با محیط شما تداخل نکند

۲. cluster لوکال Kubernetes توصیه می‌شود؛ بخش ۰۲ بدون cluster
با setup_notes.txt قابل ارزیابی است

۳. مهلت تحویل: جمعه هفته آینده

منابع

• [Kubernetes Documentation](#)

• [Docker Documentation](#)

• [kubectl Cheat Sheet](#)

موفق باشید.